

Viva: Project 06, Pokédex Lite

API: PokéAPI (<https://pokeapi.co>) - Key: No key needed

Brief: Scrollable Pokémon list with a detail screen. Great intro to pagination.

30 questions, roughly easy to hard. Sections A and B are common ground for every project; C and D are specific to this one. Ask 8 to 12 in a typical 10 minute viva.

A. Kotlin and Android basics

1. What is the difference between `val` and `var`, and which did you use for your Retrofit instance?
2. What is a data class, and what does Kotlin generate for you?
3. How did you model the list item versus the detail object, and why are they different types?
4. What does `?` mean on a property, and which detail fields are genuinely optional?
5. How do you pass the selected Pokémon from the list screen to the detail screen?
6. How does a RecyclerView adapter know what to draw for each row?
7. Where do you declare the INTERNET permission, and what happens if you forget it?

B. Retrofit, coroutines and networking

8. What problem does Retrofit solve for you?
9. Explain the role of the converter factory in your Retrofit builder.
10. Which annotations carry limit and offset to the API?
11. Why is your API call function marked suspend?
12. Which dispatcher runs the network call, and which updates the UI?
13. What happens if you call the API on the main thread?
14. What is a coroutine scope, and which one did you launch from?
15. How does your code behave if page three fails while pages one and two loaded fine?

C. The PokéAPI and this app

16. What is the exact list endpoint, and what do limit and offset do?
17. The list response gives only a name and a URL, not the details. Why did the API designers do that?
18. So how do you get a Pokémon's sprite and types? How many requests does one detail screen cost?
19. `count`, `next` and `previous` come back with every page. Which did you use to drive pagination?
20. How do you detect that the user has scrolled near the bottom and it is time to load more?
21. What stops you firing the same "load next page" request twice while one is already running?
22. Sprites come as URLs. Which image library did you use, and what does it do about caching?
23. A Pokémon can have one or two types. How does your model and your UI handle both?
24. PokéAPI asks clients to cache politely. What does that mean in practice for your app?

D. Edge cases, design and extensions

25. What does your app show while the first page is loading, and while a later page is loading?
26. A user has no connection and scrolls. Describe exactly what they see.

27. What happens if the user rotates the device after scrolling to page five?
28. How would you add search by name, given the list endpoint has no search parameter?
29. How would you cache pages locally so a second launch is instant?
30. How would you show evolution chains, which live behind two more levels of URL?

Humble Coders project viva preparation. These are the kinds of questions you should be able to answer about the app you built. Knowing the list is not the same as knowing your own code.